

KIMI K2.5: VISUAL AGENTIC INTELLIGENCE

TECHNICAL REPORT OF KIMI K2.5

Kimi Team

ABSTRACT

We introduce Kimi K2.5, an open-source multimodal agentic model designed to advance general agentic intelligence. K2.5 emphasizes the joint optimization of text and vision so that two modalities enhance each other. This includes a series of techniques such as joint text-vision pre-training, zero-vision SFT, and joint text-vision reinforcement learning. Building on this multimodal foundation, K2.5 introduces Agent Swarm, a self-directed parallel agent orchestration framework that dynamically decomposes complex tasks into heterogeneous sub-problems and executes them concurrently. Extensive evaluations show that Kimi K2.5 achieves state-of-the-art results across various domains including coding, vision, reasoning, and agentic tasks. Agent Swarm also reduces latency by up to $4.5\times$ over single-agent baselines. We release the post-trained Kimi K2.5 model checkpoint¹ to facilitate future research and real-world applications of agentic intelligence.

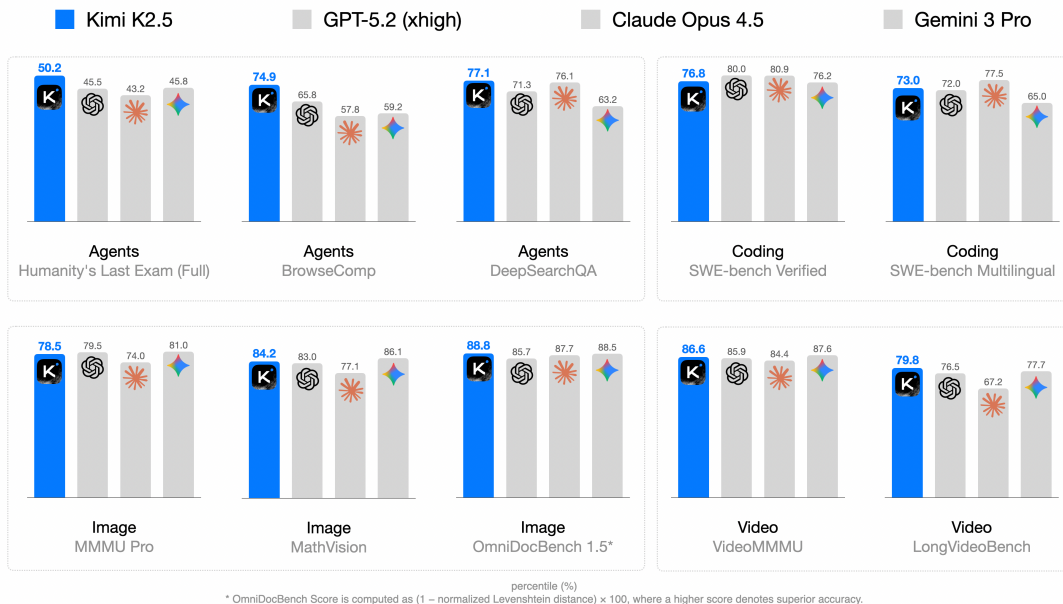


Figure 1: Kimi K2.5 main results.

1 Introduction

Large Language Models (LLMs) are rapidly evolving toward agentic intelligence. Recent advances, such as GPT-5.2 [41], Claude Opus 4.5 [6], Gemini 3 Pro [20], and Kimi K2-Thinking [1], demonstrate substantial progress in agentic capabilities, particularly in tool calling and reasoning. These models increasingly exhibit the ability to decompose complex problems into multi-step plans and to execute long sequences of interleaved reasoning and actions.

¹<https://huggingface.co/moonshotai/Kimi-K2.5>

In this report, we introduce the training methods and evaluation results of Kimi K2.5. Concretely, we improve the training of K2.5 over previous models in the following two key aspects.

Joint Optimization of Text and Vision. A key insight from the practice of K2.5 is that joint optimization of text and vision enhances both modalities and avoids the conflict. Specifically, we devise a set of techniques for this purpose. During pre-training, in contrast to conventional approaches that add visual tokens to a text backbone at a late stage [8, 21], we find early vision fusion with lower ratios tends to yield better results given the fixed total vision-text tokens. Therefore, K2.5 mixes text and vision tokens with a constant ratio throughout the entire training process.

Architecturally, Kimi K2.5 employs MoonViT-3D, a native-resolution vision encoder incorporating the NaViT packing strategy [15], enabling variable-resolution image inputs. For video understanding, we introduce a lightweight 3D ViT compression mechanism: consecutive frames are grouped in fours, processed through the shared MoonViT encoder, and temporally averaged at the patch level. This design allows Kimi K2.5 to process videos up to $4 \times$ longer within the same context window while maintaining complete weight sharing between image and video encoders.

During post-training, we introduce zero-vision SFT—text-only SFT alone activates visual reasoning and tool use. We find that adding human-designed visual trajectories at this stage hurts generalization. In contrast, text-only SFT performs better—likely because joint pretraining already establishes strong vision-text alignment, enabling capabilities to generalize naturally across modalities. We then apply joint RL on both text and vision tasks. Crucially, we find visual RL enhances textual performance rather than degrading it, with improvements on MMLU-Pro and GPQA-Diamond. This bidirectional enhancement—text bootstraps vision, vision refines text—represents superior cross-modal alignment in joint training.

Agent Swarm: Parallel Agent Orchestration. Most existing agentic models rely on sequential execution of tool calls. Even systems capable of hundreds of reasoning steps, such as Kimi K2-Thinking [1], suffer from linear scaling of inference time, leading to unacceptable latency and limiting task complexity. As agentic workloads grow in scope and heterogeneity—e.g., building a complex project that involves massive-scale research, design, and development—the sequential paradigm becomes increasingly inefficient.

To overcome the latency and scalability limits of sequential agent execution, Kimi K2.5 introduces *Agent Swarm*, a dynamic framework for parallel agent orchestration. We propose a Parallel-Agent Reinforcement Learning (PARL) paradigm that departs from traditional agentic RL [2]. In addition to optimizing tool execution via verifiable rewards, the model is equipped with interfaces for sub-agent creation and task delegation. During training, sub-agents are frozen and their execution trajectories are excluded from the optimization objective; only the orchestrator is updated via reinforcement learning. This decoupling circumvents two challenges of end-to-end co-optimization: credit assignment ambiguity and training instability. Agent Swarm enables complex tasks to be decomposed into heterogeneous sub-problems executed concurrently by domain-specialized agents, transforming task complexity from linear scaling to parallel processing. In wide-search scenarios, Agent Swarm reduces inference latency by up to $4.5 \times$ while improving item-level F1 from 72.8% to 79.0% compared to single-agent baselines.

Kimi K2.5 represents a unified architecture for general-purpose agentic intelligence, integrating vision and language, thinking and instant modes, chats and agents. It achieves strong performance across a broad range of agentic and frontier benchmarks, including state-of-the-art results in visual-to-code generation (image/video-to-code) and real-world software engineering in our internal evaluations, while scaling both the diversity of specialized agents and the degree of parallelism. To accelerate community progress toward General Agentic Intelligence, we open-source our post-trained checkpoints of Kimi K2.5, enabling researchers and developers to explore, refine, and deploy scalable agentic intelligence.

2 Joint Optimization of Text and Vision

Kimi K2.5 is a native multimodal model built upon Kimi K2 through large-scale joint pre-training on approximately 15 trillion mixed visual and text tokens. Unlike vision-adapted models that compromise either linguistic or visual capabilities, our joint pre-training paradigm enhances both modalities simultaneously. This section describes the multimodal joint optimization methodology that extends Kimi K2 to Kimi K2.5.

2.1 Native Multimodal Pre-Training

A key design question for multimodal pre-training is: Given a fixed vision-text token budget, what is the optimal vision-text joint-training strategy. Conventional wisdom [8, 21] suggests introducing vision tokens predominantly in the later stages of LLM training at high ratios (e.g., 50% or higher) should accelerate multimodal capability acquisition, treating multimodal capability as a post-hoc add-on to linguistic competence.

Table 1: Performance comparison across different vision-text joint-training strategies. Early fusion with a lower vision ratio yields better results given a fixed total vision-text token budget.

	Vision Injection Timing	Vision-Text Ratio	Vision Knowledge	Vision Reasoning	OCR	Text Knowledge	Text Reasoning	Code
Early	0%	10%:90%	25.8	43.8	65.7	45.5	58.5	24.8
Mid	50%	20%:80%	25.0	40.7	64.1	43.9	58.6	24.0
Late	80%	50%:50%	24.2	39.0	61.5	43.1	57.8	24.0

However, our experiments (as shown in Table 1 Figure 9) reveal a different story. We conducted ablation studies varying the vision ratio and vision injection timing while keeping the total vision and text token budgets fixed. To strictly meet the targets for different ratios, we pre-trained the model with text-only tokens for a specifically calculated number of tokens before introducing vision data. Surprisingly, we found that the vision ratio has minimal impact on final multimodal performance. In fact, **early fusion with a lower vision ratio yields better results given a fixed total vision-text token budget**. This motivates our native multimodal pre-training strategy: rather than aggressive vision-heavy training concentrated at the end, we adopt a moderate vision ratio integrated early in the training process, allowing the model to naturally develop balanced multimodal representations while benefiting from extended co-optimization of both modalities.

2.2 Zero-Vision SFT

Pretrained VLMs do not naturally perform vision-based tool-calling, which poses a cold-start problem for multimodal RL. Conventional approaches address this issue through manually annotated or prompt-engineered chain-of-thought (CoT) data [8], but such methods are limited in diversity, often restricting visual reasoning to simple diagrams and primitive tool manipulations (`crop`, `rotate`, `flip`).

An observation is that high-quality text SFT data are relatively abundant and diverse. We propose a novel approach, zero-vision SFT, that uses only text SFT data to activate the visual, agentic capabilities during post-training. In this approach, all image manipulations are proxied through programmatic operations in `IPython`, effectively serving as a generalization of traditional vision tool-use. This "zero-vision" activation enables diverse reasoning behaviors, including pixel-level operations such as object size estimation via binarization and counting, and generalizes to visually grounded tasks such as object localization, counting, and OCR.

Figure 2 illustrates the RL training curves, where the starting points are obtained from zero-vision SFT. The results show that zero-vision SFT is sufficient for activating vision capabilities while ensuring generalization across modalities. This phenomenon is likely due to the joint pretraining of text and vision data as described in Section 2.1. Compared to zero-vision SFT, our preliminary experiments show that text-vision SFT yields much worse performance on visual, agentic tasks, possibly because of the lack of high-quality vision data.

2.3 Joint Multimodal Reinforcement Learning (RL)

In this section, we describe the methodology implemented in K2.5 that enables effective multimodal RL, from outcome-based visual RL to emergent cross-modal transfer that enhances textual performance.

Outcome-Based Visual RL Following the zero-vision SFT, the model requires further refinement to reliably incorporate visual inputs into reasoning. Text-initiated activation alone exhibits notable failure modes: visual inputs are sometimes ignored, and images may not be attended to when necessary. We employ outcome-based RL on tasks that explicitly require visual comprehension for correct solutions. We categorize these tasks into three domains:

- **Visual grounding and counting:** Accurate localization and enumeration of objects within images;
- **Chart and document understanding:** Interpretation of structured visual information and text extraction;
- **Vision-critical STEM problems:** Mathematical and scientific questions filtered to require visual inputs.

Outcome-based RL on these tasks improves both basic visual capabilities and more complex agentic behaviors. Extracting these trajectories for rejection-sampling fine-tuning (RFT) enables a self-improving data pipeline, allowing subsequent joint RL stages to leverage richer multimodal reasoning traces.

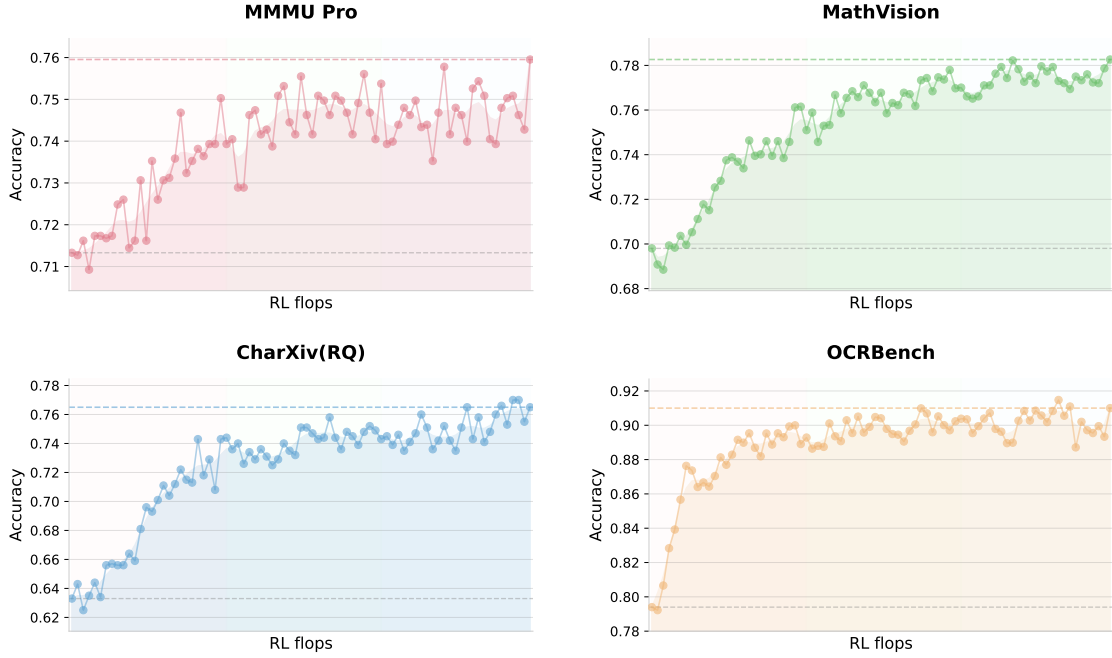


Figure 2: Vision RL training curves on vision benchmarks starting from minimal zero-vision SFT. By scaling vision RL FLOPs, the performance continues to improve, demonstrating that zero-vision activation paired with long-running RL is sufficient for acquiring robust visual capabilities.

Table 2: Cross-Modal Transfer: Vision RL Improves Textual Knowledge

Benchmark	Before Vision-RL	After Vision-RL	Improvement
MMLU-Pro	84.7	86.4	+1.7
GPQA-Diamond	84.3	86.4	+2.1
LongBench v2	56.7	58.9	+2.2

Visual RL Improves Text Performance To investigate potential trade-offs between visual and textual performance, we evaluated text-only benchmarks before and after visual RL. Surprisingly, outcome-based visual RL produced measurable improvements in textual tasks, including MMLU-Pro (84.7% $\rightarrow$ 86.4%), GPQA-Diamond (84.3% $\rightarrow$ 86.4%), and LongBench v2 (56.7% $\rightarrow$ 58.9%) (Table 2). Analysis suggests that visual RL enhances calibration in areas requiring structured information extraction, reducing uncertainty on queries that resemble visually grounded reasoning (e.g., counting, OCR). These findings indicate that visual RL can contribute to cross-modal generalization, improving textual reasoning without observable degradation of language capabilities.

Joint Multimodal RL Motivated by the finding that robust visual capabilities can emerge from zero-vision SFT paired with vision RL—which further enhances general text abilities—we adopt a joint multimodal RL paradigm during Kimi K2.5’s post-training. Departing from conventional modality-specific expert divisions, we organize RL domains not by input modality but by abilities—knowledge, reasoning, coding, agentic, etc. These domain experts jointly learn from both pure-text and multimodal queries, while the Generative Reward Model (GRM) similarly optimizes across heterogeneous traces without modality barriers. This paradigm ensures that capability improvements acquired through either textual or visual inputs inherently generalize to enhance related abilities across the alternate modality, thereby maximizing cross-modal capability transfer.

3 Agent Swarm

The primary challenge of existing agent-based systems lies in their reliance on sequential execution of reasoning and tool-calling steps. While this structure may be effective for simpler, short-horizon tasks, it becomes inadequate as the complexity of the task increases and the accumulated context grows. As tasks evolve to contain broad information gathering and intricate, multi-branch reasoning, sequential systems often encounter significant bottlenecks [5, 6, 7].

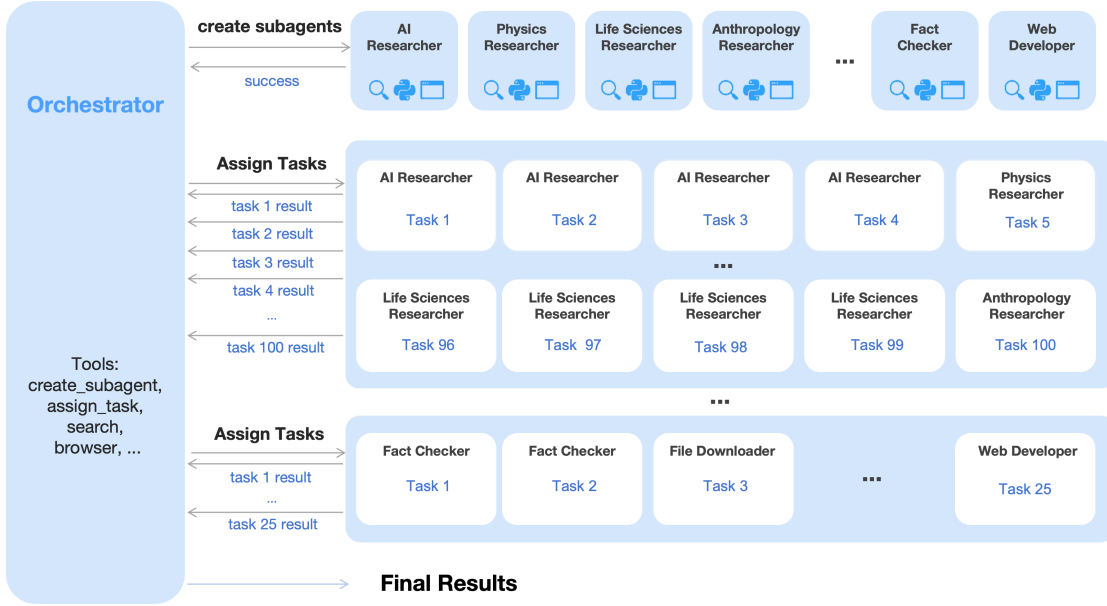


Figure 3: An agent swarm has a trainable orchestrator that dynamically creates specialized frozen subagents and decomposes complex tasks into parallelizable subtasks for efficient distributed execution.

The limited capacity of a single agent working through each step one by one can lead to the exhaustion of practical reasoning depth and tool-call budgets, ultimately hindering the system’s ability to handle more complex scenarios.

To address this, we introduce **Agent Swarm** and **Parallel Agent Reinforcement Learning (PARL)**. Instead of executing a task as a reasoning chain or relying on pre-specified parallelization heuristics, K2.5 initiates an Agent Swarm through dynamic task decomposition, subagent instantiation, and parallel subtask scheduling. Importantly, parallelism is not presumed to be inherently advantageous; decisions regarding whether, when, and how to parallelize are explicitly learned through environmental feedback and RL-driven exploration. As shown in Figure 4, the progression of performance demonstrates this adaptive capability, with the cumulative reward increasing smoothly as the orchestrator optimizes its parallelization strategy throughout training.

Architecture and Learning Setup The PARL framework adopts a decoupled architecture comprising a trainable orchestrator and frozen subagents instantiated from fixed intermediate policy checkpoints. This design deliberately avoids end-to-end co-optimization to circumvent two fundamental challenges: credit assignment ambiguity and training instability. In this multi-agent setting, outcome-based rewards are inherently sparse and noisy; a correct final answer does not guarantee flawless subagent execution, just as a failure does not imply universal subagent error. By freezing the subagents and treating their outputs as environmental observations rather than differentiable decision points, we disentangle high-level coordination logic from low-level execution proficiency, leading to more robust convergence. To improve efficiency, we first train the orchestrator using small-size subagents before transitioning to larger models. Our RL framework also supports dynamically adjusting the inference instance ratios between subagents and the orchestrator, thereby maximizing the resource usage across the cluster.

PARL Reward Training a reliable parallel orchestrator is challenging due to the delayed, sparse, and non-stationary feedback inherent in independent subagent execution. To address this, we define the PARL reward as:

$$r_{\text{PARL}}(x, y) = \underbrace{\lambda_1 \cdot r_{\text{parallel}}}_{\text{instantiation reward}} + \underbrace{\lambda_2 \cdot r_{\text{finish}}}_{\text{sub-agent finish rate}} + \underbrace{r_{\text{perf}}(x, y)}_{\text{task-level outcome}}.$$

The performance reward r_{perf} evaluates the overall success and quality of the solution y for a given task x . This is augmented by two auxiliary rewards, each addressing a distinct challenge in learning parallel orchestration. The reward r_{parallel} is introduced to mitigate *serial collapse*—a local optimum where the orchestrator defaults to single-agent execution. By incentivizing subagent instantiation, this term encourages the exploration of concurrent scheduling spaces. The r_{finish} reward focuses on the successful completion of assigned subtasks. It is used to prevent *spurious parallelism*, a reward-hacking behavior in which the orchestrator increases parallel metrics dramatically by spawning